

1.实验目的

- (1) 了解汇编语言程序(源程序)的基本组成部分
- (2) 熟悉 CPU 中各个寄存器的作用
- (3) 进一步掌握使用 DEBUG 相应命令进行程序修改以及指令的调试与运行。

2.实验环境

通过 DOSBox 模拟器，在 64 位操作系统运行 DOS 环境，作为汇编语言程序的开发与运行平台。

3.实验内容

1.将给定程序输入，并汇编、连接后生成可执行文件lab2.exe

2.Debug lab2.exe

- 将内存中字符串“hello world”改写成“HELLO WORLD”，并显示修改后的结果
- 展示3F24+4A2B和3F24 - 4A2B的计算
- 在内存中输入 MOV AX, 50

ADD AX,AX

执行并查看AX的变化，修改AX的值为FFFF

3.尝试下文提到的DEBUG的各个选项

- R 查看、修改寄存器内容
- D 查看内存内容
- E 修改内存内容
- U 将内存中机器指令翻译成汇编指令
- A 以汇编指令的格式在内存中写入机器指令
- T 执行一条机器指令
- P 执行一个子程序或循环指令
- Q 退出
- ? 使用帮助

4.实验具体实现

1.汇编并 链接生成可执行文件lab2.exe

源代码内容

ASM lab2.ASM

```
1  data segment
2      string DB 'hello world!!!', 0AH, 0DH, '$'
3  data ends
4
5  code segment
6      assume cs:code, ds:data
7  start:
8      MOV AX, DATA
9      MOV DS, AX
10
11      mov dx, OFFSET string
12      mov AH, 09H
13      int 21H
14
15      MOV AH, 4CH
16      INT 21H
17  code ends
18  end start
19
```

编译和运行结果

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

D:\>ml /c lab2.asm
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

    Assembling: lab2.asm

D:\>link lab2.obj

Microsoft (R) Segmented Executable Linker Version 5.60.339 Dec  5 1994
Copyright (C) Microsoft Corp 1984-1993. All rights reserved.

Run File [lab2.exe]:
List File [nul.map]:
Libraries [.lib]:
Definitions File [nul.def]:
LINK : warning L4021: no stack segment

D:\>lab2.exe
hello world!!!

D:\>_
```

使用mk汇编后link为可执行程序

2.DEBUG lab2.exe

1.将hello world改为HELLO WORLD

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
D:\>debug lab2.exe
-d 0100
075A:0100 68 65 6C 6C 6F 20 77 6F-72 6C 64 21 21 21 0A 0D hello world!!!..
075A:0110 24 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 $.
075A:0120 B8 6A 07 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .j.....!.L.!
075A:0130 56 D0 3B 56 1A 75 03 3B-46 18 76 06 89 46 18 89 U.;U.u.;F.v..F..
075A:0140 56 1A B8 04 00 50 0E E8-A6 0A B8 81 27 50 FF 76 U...P.....'P.v
075A:0150 1A FF 76 18 B8 F2 52 50-0E E8 9E 08 83 C4 0A A1 ..v...RP.....
075A:0160 F8 56 0B 06 FA 56 74 1B-B8 08 00 50 0E E8 80 0A .U...Ut....P....
075A:0170 FF 36 FA 56 FF 36 F8 56-B8 FA 52 50 0E E8 7A 08 .6.U.6.U..RP..z.
-e 0100
075A:0100 68.48 65.45 6C.4C 6C.4C 6F.4F 20. 77.57 6F.
075A:0108 72.52 6C.4C 64.44 21. 21. 21.
-d 0100
075A:0100 48 45 4C 4C 4F 20 57 6F-52 4C 44 21 21 21 0A 0D HELLO World!!!..
075A:0110 24 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 $.
075A:0120 B8 6A 07 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .j.....!.L.!
075A:0130 56 D0 3B 56 1A 75 03 3B-46 18 76 06 89 46 18 89 U.;U.u.;F.v..F..
075A:0140 56 1A B8 04 00 50 0E E8-A6 0A B8 81 27 50 FF 76 U...P.....'P.v
075A:0150 1A FF 76 18 B8 F2 52 50-0E E8 9E 08 83 C4 0A A1 ..v...RP.....
075A:0160 F8 56 0B 06 FA 56 74 1B-B8 08 00 50 0E E8 80 0A .U...Ut....P....
075A:0170 FF 36 FA 56 FF 36 F8 56-B8 FA 52 50 0E E8 7A 08 .6.U.6.U..RP..z.
```

在第一次修改时不小心漏修改了一个字母，后面补上了

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
075A:0100 48 45 4C 4C 4F 20 57 6F-52 4C 44 21 21 21 0A 0D HELLO World!!!..
075A:0110 24 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 $.
075A:0120 B8 6A 07 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .j.....!.L.!
075A:0130 56 D0 3B 56 1A 75 03 3B-46 18 76 06 89 46 18 89 U.;U.u.;F.v..F..
075A:0140 56 1A B8 04 00 50 0E E8-A6 0A B8 81 27 50 FF 76 U...P.....'P.v
075A:0150 1A FF 76 18 B8 F2 52 50-0E E8 9E 08 83 C4 0A A1 ..v...RP.....
075A:0160 F8 56 0B 06 FA 56 74 1B-B8 08 00 50 0E E8 80 0A .U...Ut....P....
075A:0170 FF 36 FA 56 FF 36 F8 56-B8 FA 52 50 0E E8 7A 08 .6.U.6.U..RP..z.
-e 0107
075A:0107 6F.
075A:0108 52. 4C. 44. 21.
-e 0107
075A:0107 6F.4F
-d 0100
075A:0100 48 45 4C 4C 4F 20 57 4F-52 4C 44 21 21 21 0A 0D HELLO WORLD!!!..
075A:0110 24 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 $.
075A:0120 B8 6A 07 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21 .j.....!.L.!
075A:0130 56 D0 3B 56 1A 75 03 3B-46 18 76 06 89 46 18 89 U.;U.u.;F.v..F..
075A:0140 56 1A B8 04 00 50 0E E8-A6 0A B8 81 27 50 FF 76 U...P.....'P.v
075A:0150 1A FF 76 18 B8 F2 52 50-0E E8 9E 08 83 C4 0A A1 ..v...RP.....
075A:0160 F8 56 0B 06 FA 56 74 1B-B8 08 00 50 0E E8 80 0A .U...Ut....P....
075A:0170 FF 36 FA 56 FF 36 F8 56-B8 FA 52 50 0E E8 7A 08 .6.U.6.U..RP..z.
```

使用e指令对于内存进行修改

```
-h 3f24 4a2b
b94f f4f9
-g
HELLO WORLD!!!

Program terminated normally
-q
```

使用g指令全速运行
程序运行结束正常输出大写的HELLO WORLD

2.展示两个计算

```
073f:0100 ff 36 fa 56 ff 36 f8 56-b8 fa 52 50 0e e8 7a 08 .6.U.6.U..RP..Z.
-h 3f24 4a2b
b94f f4f9
-
```

如图由h指令的定义可知第一个数为3f24+4a2b的结果，第二个数为3f24-4a2b的结果

3.在内存中输入汇编指令并且修改AX的值为0xFFFF

1.运行两条汇编指令

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
D:\>debug
-a
073f:0100 MOV AX,50
073f:0103 ADD AX,AX
073f:0105
-t
AX=0050 BX=0000 CX=0000 DX=0000 SP=00fd BP=0000 SI=0000 DI=0000
DS=073f ES=073f SS=073f CS=073f IP=0103 NU UP EI PL NZ NA PO NC
073f:0103 01c0 ADD AX,AX
-t
AX=00a0 BX=0000 CX=0000 DX=0000 SP=00fd BP=0000 SI=0000 DI=0000
DS=073f ES=073f SS=073f CS=073f IP=0105 NU UP EI PL NZ NA PE NC
073f:0105 0000 ADD [BX+SI],AL DS:0000=cd
-t
AX=00a0 BX=0000 CX=0000 DX=0000 SP=00fd BP=0000 SI=0000 DI=0000
DS=073f ES=073f SS=073f CS=073f IP=0107 OU UP EI PL NZ NA PO CY
073f:0107 0000 ADD [BX+SI],AL DS:0000=6d
-r
AX=00a0 BX=0000 CX=0000 DX=0000 SP=00fd BP=0000 SI=0000 DI=0000
DS=073f ES=073f SS=073f CS=073f IP=0107 OU UP EI PL NZ NA PO CY
073f:0107 0000 ADD [BX+SI],AL DS:0000=6d
-
```

先使用a指令往内存中写入指令
然后用t指令逐条执行查看寄存器变化
最后用r指令查看最终的寄存器值
可以看到刚开始把AX寄存器赋值为0x50，然后自增后AX寄存器变为0xA0

2.修改AX为0xFFFF

```
-r
AX=00A0 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0107  OV UP EI PL NZ NA PO CY
073F:0107 0000          ADD     [BX+SI],AL          DS:0000=6D
-r ax
AX 00A0
:FFFF
-r
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0107  OV UP EI PL NZ NA PO CY
073F:0107 0000          ADD     [BX+SI],AL          DS:0000=6D
```

使用r指令把AX修改为0xFFFF

3.使用DEBUG指令

1.r指令

```
-r
AX=FFFF BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=0769 CS=076C IP=0000  NU UP EI PL NZ NA PO NC
076C:0000 B86A07          MOV     AX,076A
-r ax
AX FFFF
:0000
-r
AX=0000 BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=0769 CS=076C IP=0000  NU UP EI PL NZ NA PO NC
076C:0000 B86A07          MOV     AX,076A
```

先使用r指令查看内存中寄存器值，然后修改ax为0x0000

2.d指令

```
-d 0100
075A:0100  68 65 6C 6C 6F 20 77 6F-72 6C 64 21 21 21 0A 0D  hello world!!!..
075A:0110  24 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  $.
075A:0120  B8 6A 07 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21  .j.....!.L.!
075A:0130  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0140  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0150  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0160  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0170  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
```

查看内存中0100的内容

3.e指令

```
-e 0100
075A:0100  68.48

-d 0100
075A:0100  48 65 6C 6C 6F 20 77 6F-72 6C 64 21 21 21 0A 0D  Hello world!!!..
075A:0110  24 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  $.....
075A:0120  B8 6A 07 8E D8 BA 00 00-B4 09 CD 21 B4 4C CD 21  .j.....!.L.!
075A:0130  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0140  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0150  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0160  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
075A:0170  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  .....
```

修改内存中0100为0x48，将小写的h改为大写的H

4.u指令

```
-u 0000
076C:0000 B86A07      MOV     AX,076A
076C:0003 8ED8      MOV     DS,AX
076C:0005 BA0000      MOV     DX,0000
076C:0008 B409      MOV     AH,09
076C:000A CD21      INT     21
076C:000C B44C      MOV     AH,4C
076C:000E CD21      INT     21
076C:0010 0000      ADD     [BX+SI],AL
076C:0012 0000      ADD     [BX+SI],AL
076C:0014 0000      ADD     [BX+SI],AL
076C:0016 0000      ADD     [BX+SI],AL
076C:0018 0000      ADD     [BX+SI],AL
076C:001A 0000      ADD     [BX+SI],AL
076C:001C 0000      ADD     [BX+SI],AL
076C:001E 0000      ADD     [BX+SI],AL
_
```

可以看到反汇编出来的结果和我们的汇编源程序几乎相同

```
data segment
    string DB 'hello world!!!', 0AH, 0DH, '$'
data ends

code segment
    assume cs:code, ds:data
start:
    MOV AX, DATA
    MOV DS, AX

    mov dx, OFFSET string
    mov AH, 09H
    int 21H

    MOV AH, 4CH
    INT 21H
code ends
end start
```


5.a指令

```
-a
076C:0000 MOV AX,FFFF
076C:0003
-u 0000
076C:0000 B8FFFF      MOV     AX,FFFF
076C:0003 8ED8        MOV     DS,AX
076C:0005 BA0000      MOV     DX,0000
076C:0008 B409        MOV     AH,09
076C:000A CD21        INT     21
076C:000C B44C        MOV     AH,4C
076C:000E CD21        INT     21
076C:0010 0000        ADD     [BX+SI],AL
076C:0012 0000        ADD     [BX+SI],AL
076C:0014 0000        ADD     [BX+SI],AL
076C:0016 0000        ADD     [BX+SI],AL
076C:0018 0000        ADD     [BX+SI],AL
076C:001A 0000        ADD     [BX+SI],AL
076C:001C 0000        ADD     [BX+SI],AL
076C:001E 0000        ADD     [BX+SI],AL
```

可以看到将第一条指令修改为修改寄存器为0xFFFF之后反汇编结果也改变了

6.t指令

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
076C:0000 MOV AX,FFFF
076C:0003
-t
AX=FFFF BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=0769 CS=076C IP=0003  NU UP EI PL NZ NA PO NC
076C:0003 8ED8        MOV     DS,AX
-t
AX=FFFF BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=FFFF ES=075A SS=0769 CS=076C IP=0005  NU UP EI PL NZ NA PO NC
076C:0005 BA0000      MOV     DX,0000
-t
AX=FFFF BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=FFFF ES=075A SS=0769 CS=076C IP=0008  NU UP EI PL NZ NA PO NC
076C:0008 B409        MOV     AH,09
-y
^ Error
-t
AX=09FF BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=FFFF ES=075A SS=0769 CS=076C IP=000A  NU UP EI PL NZ NA PO NC
076C:000A CD21        INT     21
```

可以看到寄存器就按照汇编指令的要求随着执行逐步改变

刚刚修改的0xFFFF也成功了

7.p指令

```
-p
AX=076A BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076C IP=0005  NU UP EI PL NZ NA PO NC
076C:0005 BA0000          MOV     DX,0000
-p
AX=076A BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076C IP=0008  NU UP EI PL NZ NA PO NC
076C:0008 B409          MOV     AH,09
-p
AX=096A BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076C IP=000A  NU UP EI PL NZ NA PO NC
076C:000A CD21          INT     21
-p
hello world!!!
AX=096A BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076C IP=000C  NU UP EI PL NZ NA PO NC
076C:000C B44C          MOV     AH,4C
-
```

p指令在执行systemcall等的时候会直接执行完而不是像t指令一样深入进去

下图是t指令的对比图

```
-t
AX=076A BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076C IP=0008  NU UP EI PL NZ NA PO NC
076C:0008 B409          MOV     AH,09
-t
AX=096A BX=0000 CX=0030 DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076C IP=000A  NU UP EI PL NZ NA PO NC
076C:000A CD21          INT     21
-t
AX=096A BX=0000 CX=0030 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=F000 IP=14A0  NU UP DI PL NZ NA PO NC
F000:14A0 FB          STI
-t
AX=096A BX=0000 CX=0030 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=F000 IP=14A1  NU UP EI PL NZ NA PO NC
F000:14A1 FE38          ???     [BX+SI]          DS:0000=68
-
```

8.q指令

```
AX=096A BX=0000 CX=0030 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=F000 IP=14A1  NU UP EI PL NZ NA PO NC
F000:14A1 FE38          ???     [BX+SI]          DS:0000=68
-q
D:\>
```

使用q指令退出调试

5.实验分析与总结

通过本次实验，让我对汇编语言程序的基本结构以及寄存器的作用有了更加直观的理解。实验过程中使用 DOSBox 模拟器和 DEBUG 工具对程序进行了多次修改与调试，加深了我对汇编寻址和赋值操作的认识。

在具体操作中，通过将“hello world”修改为“HELLO WORLD”，我更加熟悉了内存修改和数据表示的方法；通过计算 $3F24+4A2B$ 和 $3F24-4A2B$ 的结果，掌握了十六进制数的运算过程；在输入并执行 MOV 和 ADD 指令时，观察了 AX 寄存器值的变化，并尝试手动修改为 FFFF，这让我对寄存器值的存储和更新过程有了直观体验。

同时，熟悉了 DEBUG 的常用指令（如 r、d、e、u、a、t、p、q），不仅能够查看和修改寄存器与内存内容，还能反汇编、逐条或逐过程执行程序，进一步提高了我对汇编程序调试方法的掌握。

总而言之，本次实验不仅提升了我对汇编语言的理解，也让我更加熟练地掌握了调试技巧，为后续学习和编写更复杂的汇编程序打下了基础。